CostGuard: Policy-Governed Cost Anomaly Intelligence for Cloud DevOps Pipelines

Tejas Sudhakar Kamble

***Abstract*—Cloud delivery pipelines frequently detect cost anomalies too late, after spend has already propagated through dependent deployment stages. This paper presents CostGuard, a production-oriented DevOps/FinOps system that connects anomaly detection to policy-enforced runtime control. CostGuard combines two modeling perspectives: a residual attention-enhanced LSTM branch for temporal stage behavior and a GATv2 branch for dependency-aware graph reasoning. Their outputs are combined by validation-calibrated adaptive ensembling and exposed to a governance layer implemented with Open Policy Agent (OPA). Evaluation follows an immutable ten-seed protocol (42–132) across three domains in lifelong order: synthetic (D0), TravisTorrent (L1), and BitBrains (L2). Mean ensemble F1/PR-AUC reaches 0.8962/0.9648 (D0), 0.9087/0.9419 (L1), and 0.9606/0.9763 (L2), with low variance in real domains (standard deviation 0.0011 and 0.0017). The full 10/10 run set was completed under an RTX 3050 4-GB profile using memory-safe controls (30 epochs, patience 5, adaptive batching, bounded preprocessing, and VRAM flush checkpoints). Results show that cost anomaly intelligence is materially more useful when coupled with policy-as-code decision gates and traceable execution evidence.**

***Index Terms*—FinOps, DevOps, CI/CD, anomaly detection, LSTM, GATv2, Open Policy Agent, policy-as-code, cloud cost governance.**

I. Introduction

DEVOPS organizations optimize speed and reliability aggressively, yet cost governance in most CI/CD stacks remains retrospective. Existing systems treat cost as a retrospective signal, limiting actionable intervention during pipeline execution. Pipelines can trigger expensive build, test, image, and deployment paths within minutes, while actionable financial feedback often appears only after the release decision window has closed.

Current FinOps platforms improve visibility and budget accountability, but many remain weakly coupled to runtime delivery control. In contrast, release orchestration systems execute continue/pause/block decisions in real time, often without model-driven cost-risk evidence. This disconnect between analytics and control is the exact operational gap that CostGuard addresses.

CostGuard embeds cost-risk intelligence into the delivery control loop itself. Instead of producing only post hoc dashboards, it attaches anomaly scoring and policy evaluation to the same decision path that determines whether a pipeline should continue, warn, auto-optimize, or block.

Tejas Sudhakar Kamble is with the Department of Computer Science, Sir Parashurambhau College, Pune, India (e-mail: tejasuskamble@gmail.com).

A preprint version of this work is available on arXiv.

This paper focuses on evidence-backed engineering and evaluation rather than architecture-only claims. The contribution is threefold.

- **Built system:** A deployable full-stack CostGuard pipeline integrating dual-view anomaly inference, OPA-governed policy gating, persistent audit records, and operational dashboards in one execution loop.
- **Proven behavior:** Under an immutable ten-seed protocol across D0/L1/L2, CostGuard reports stable high real-domain performance (ensemble F1 0.9087 in L1 and 0.9606 in L2, standard deviation 0.0011 and 0.0017), replacing single-run claims with reproducible evidence.
- **Practical relevance:** A laptop-constrained reliability profile (RTX 3050 4-GB, 10/10 completed runs) demonstrates that policy-governed cost intervention is feasible with traceable artifacts on commodity engineering hardware.

II. Related Work

A. Sequence and Graph Modeling

LSTM remains a standard method for learning delayed dependencies in event sequences [2]. This is directly relevant to cost-risk behavior, where an expensive event may surface several stages later. Graph Attention Networks model non-uniform influence among connected entities [3], matching pipeline structures where upstream stages do not contribute equally to downstream cost escalation. Additional work on latent network inference and weakly supervised structure parsing further supports relational modeling when links are noisy or partially observed [4], [5]. Unlike prior modeling studies that evaluate temporal or relational quality in isolation, CostGuard evaluates these views inside a runtime governance loop.

B. FinOps and Cost Optimization

Recent cloud cost literature highlights automation, observability, and budget-aware controls [11], [12]. OPTIC extends this direction through FOCUS-aligned schema normalization and user-facing cloud-cost queryability [16]. In contrast to CostGuard, these efforts primarily emphasize analytics interoperability and reporting, with limited seed-controlled cross-domain ML evidence tied to runtime release control actions.

C. CI/CD Optimization and Governance

ML-assisted CI/CD studies report improvements in delivery efficiency and quality [6]–[10]. Governance-oriented work emphasizes policy-bounded automation and elasticity-aware

decisions [13], [15]. Unlike prior work, integrated systems that jointly expose (i) reproducible model behavior, (ii) explicit policy outcomes, and (iii) deployment-grade artifact provenance remain limited.

D. Research Gap

The unresolved gap is integration depth. Prior work commonly optimizes one layer—modeling, cost analytics, or policy orchestration—while production DevOps/FinOps adoption requires all three to operate coherently. This limitation motivates an end-to-end system where anomaly inference, policy actuation, and reproducibility artifacts are evaluated together. Table I summarizes this positioning.

TABLE I
POSITIONING PRIOR LITERATURE AGAINST INTEGRATED CI/CD COST-GOVERNANCE REQUIREMENTS.

Theme	Representative studies	Remaining gap addressed by CostGuard
Sequence modeling	LSTM foundations [2]	Strong temporal memory, but no policy actioning or deployment governance semantics.
Graph dependency modeling	GAT and related graph inference studies [3]–[5]	Strong relational reasoning, but no fixed-seed CI/CD cost-risk evidence loop.
FinOps optimization strategy	Cloud cost reviews and ABACUS-style services [11], [12]	Practical governance direction, but limited reproducible multi-domain ML validation.
CI/CD acceleration with ML	Pipeline optimization studies [6]–[10]	Better delivery performance, but weak linkage between anomaly scores and enforceable runtime policy outcomes.
Policy-governed cloud operations	Agentic and elasticity policy work [13], [15]	Rich governance concepts, but limited integration with cross-domain anomaly benchmarking and immutable seed artifacts.
Cost schema standardization	OPTIC and FOCUS-oriented cost analytics [16], [18]	Better cost interoperability, but no direct coupling to CI/CD risk gates and release control decisions.

III. METHODOLOGY

A. Dual-View Anomaly Modeling

CostGuard implements two complementary predictors.

- *Temporal branch (LSTM):* Bahdanau-attention residual BiLSTM stack over stage-wise telemetry sequences.
- *Dependency branch (GATv2):* multi-layer graph attention model over pipeline dependency graphs.

The LSTM branch captures temporal carry-over behavior in CI/CD runs, where the economic effect of one stage can appear several stages later. The GATv2 branch captures structural propagation, allowing risk at shared predecessors to influence downstream decisions with learned edge importance. Together, they represent complementary failure modes: sequence drift and dependency spillover.

The ensemble score is defined as

$$\hat{p}_{ens} = w_L \hat{p}_{LSTM} + w_G \hat{p}_{GAT}, \tag{1}$$

with validation-derived weights

$$w_i = \frac{\text{PR-AUC}_i}{\text{PR-AUC}_{LSTM} + \text{PR-AUC}_{GAT}}. \tag{2}$$

Weights are frozen after validation to avoid test leakage.

B. Adaptive Fusion for Robust Generalization

Ensembling is adaptive, not unconditional. If graph inputs are missing or branch calibration is unfavorable, inference switches to `lstm_only` mode. This matters in practical pipelines, where graph completeness and dependency semantics vary across repositories and environments. Weighted fusion is used when both branches are calibrated; otherwise, single-branch routing prevents a weaker branch from degrading release-time decisions. This selective control is central to cross-domain robustness.

C. Threshold Selection and Leakage Control

For each domain-seed run, thresholds are tuned on validation logits and then locked for test inference. This protocol avoids post-test threshold drift. Scalers are fit only on training partitions and reused on validation/test partitions, preventing data leakage from artificial calibration gains. Together with seed-isolated workspaces and immutable manifest writing, these controls enforce consistent reproducibility behavior across the full ten-seed run set.

D. CRS and Policy Mapping

Cost Risk Score (CRS) is the policy-facing risk scalar consumed by the governance layer. In the current backend runtime, CRS is computed from the selected model risk signal (default API behavior maps CRS from the GAT probability path) and evaluated with operational context: stage identity, branch state, pull-request flags, and billed stage cost.

OPA policies map CRS and context to four actions: `ALLOW`, `WARN`, `AUTO_OPTIMISE`, and `BLOCK`. OPA-first evaluation is paired with deterministic inline fallback, preserving service continuity while recording policy provenance through a `policy_source` field. The full control flow from telemetry to governed action is shown in Fig. 1.

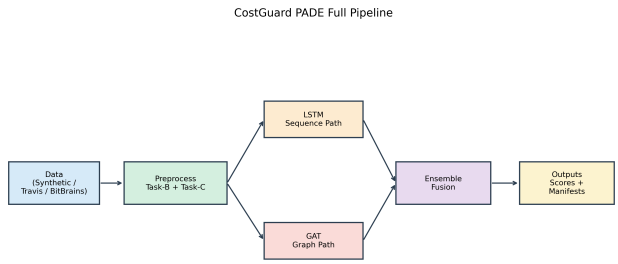

Fig. 1. CostGuard control loop from telemetry ingestion to policy-governed actioning.

IV. SYSTEM ARCHITECTURE

The deployment stack includes: (1) training/inference engine, (2) FastAPI backend, (3) PostgreSQL persistence, (4) OPA policy service, and (5) Streamlit operational dashboards. Architecture decisions were driven by traceability requirements rather than benchmark-only performance.

A. Evidence Persistence

CostGuard stores per-run and per-stage records that include thresholds, selected strategy, decision class, matched policy rules, and remediation text. This allows auditors to reconstruct the path from raw model score to final control decision.

B. Governance Layer Behavior

Default policy thresholds are 0.50 (warn), 0.75 (auto-optimize), and 0.90 (block). Rule bundles also include contextual gates (for example, blocking production deploys from

pull-request contexts and enforcing sensitive-stage restrictions on protected branches). Table II lists the deployed rule behavior.

TABLE II
POLICY BUNDLE BEHAVIOR USED BY THE GOVERNANCE LAYER.

Rule type	Runtime effect
Threshold gates	Escalate action class at CRS $\geq$ 0.50, 0.75, 0.90
Context gates	Block risky branch/stage combinations regardless of low CRS
Stage cost ceilings	Trigger WARN/AUTO_OPTIMISE when billed cost crosses stage-specific limits
OPA fallback safety	Preserve deterministic policy decisioning when OPA endpoint is unavailable

V. EXPERIMENTAL SETUP

A. Protocol and Domain Order

All experiments follow D0→L1→L2 using canonical seeds {42, 52, 62, 72, 82, 92, 102, 112, 122, 132}. Aggregate metadata confirms 10 completed trials out of 10, and the dataset template is reported in Table III.

TABLE III
DATASET SETUP AND SPLIT SIZES (SEED 42 CANONICAL TEMPLATE).

Domain	Train	Val	Test
D0 Synthetic (Task-B)	43,750	9,375	9,375
L1 TravisTorrent (Task-B)	60,279	12,917	12,918
L2 BitBrains (Task-B)	174,365	37,364	37,364

Task-C graph partitions were also generated for all three domains (for example, L2 graph split: 34,582/7,410/7,411 train/val/test).

B. Metrics and Thresholding

Primary metrics are F1 at validation-optimized threshold and PR-AUC. PR-AUC is treated as deployment-critical because anomalies are class-imbalanced and high-recall operation is required for cost containment.

C. Hardware-Constrained Reproducibility Design

Execution used an RTX 3050 Laptop GPU profile (4095 MB VRAM reported), 30 epochs, learning rate 5×10^{-4}, and patience 5. The run engine enforced memory-safe design via adaptive batch downshift, guarded mixed precision, bounded CSV materialization, and explicit VRAM flush checkpoints between phases. Core runtime settings are summarized in Table IV, and implementation trade-offs are listed in Table V.

TABLE IV
RUNTIME PROFILE USED FOR ALL REPORTED RUNS.

Setting	Value
GPU profile	NVIDIA RTX 3050 Laptop, 4095 MB VRAM
Training horizon	30 epochs (laptop-safe mode)
Early stopping	Patience = 5
Learning rate	5×10^{-4}
Seed protocol	10 fixed seeds (42–132)
Memory safety	Adaptive batching + bounded preprocessing + VRAM flush

TABLE V
ENGINEERING TRADE-OFFS USED TO PRESERVE RUN CONTINUITY AND REPRODUCIBILITY.

Constraint	Design decision
4-GB VRAM budget	Conservative batch ladders for LSTM and GAT with automatic downshift on pressure.
Long multi-seed runs	Early stopping (patience 5) to limit wasted epochs after convergence.
Large raw CSV sources	Bounded chunked preprocessing to avoid unsafe full materialization.
Mixed precision risk	Enable fp16 only with sufficient free VRAM headroom.
Cross-seed contamination risk	Seed-isolated workspaces and run-specific manifests for every domain stage.
Policy availability risk	OPA-first evaluation with deterministic inline fallback and source tagging.

VI. RESULTS AND ANALYSIS

A. Aggregate Model Performance

Table VI and Fig. 2 report means over ten seeds. Ensemble behavior is strong across all domains, with the best separability in L2 BitBrains. A likely contributor is the larger L2 training pool and more stable stage-level execution patterns, which enable a lower operating threshold (0.180) while preserving very high recall.

TABLE VI
AGGREGATE METRICS BY DOMAIN AND MODEL (MEAN OVER 10 SEEDS).

Domain	Model	F1@opt	ROC-AUC	PR-AUC	Precision	Recall	Threshold
Synthetic (D0)	LSTM	0.9128	0.9526	0.9766	0.9326	0.8941	0.2940
Synthetic (D0)	GAT	0.7468	0.7646	0.8209	0.7104	0.7932	0.3790
Synthetic (D0)	ENS	0.8962	0.9354	0.9648	0.9139	0.8799	0.3900
TravisTorrent (L1)	LSTM	0.9087	0.8133	0.9419	0.8454	0.9825	0.3710
TravisTorrent (L1)	GAT	0.7683	0.6939	0.8274	0.6246	0.9981	0.3370
TravisTorrent (L1)	ENS	0.9087	0.8133	0.9419	0.8454	0.9825	0.3710
BitBrains (L2)	LSTM	0.9606	0.9628	0.9763	0.9271	0.9966	0.1800
BitBrains (L2)	GAT	0.8760	0.5905	0.8281	0.7800	0.9990	0.2480
BitBrains (L2)	ENS	0.9606	0.9628	0.9763	0.9271	0.9966	0.1800

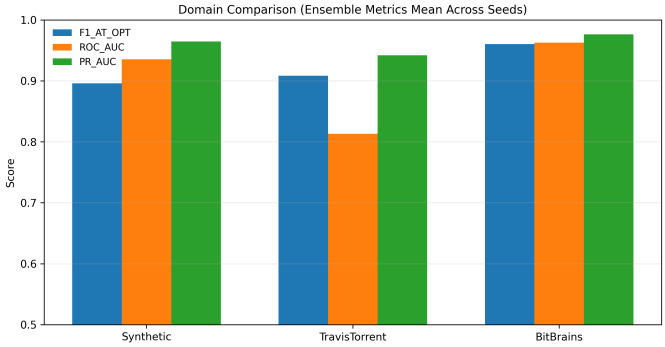

Fig. 2. Domain-wise metric comparison for LSTM, GAT, and ensemble outputs.

B. Model Delta Analysis

Table VII separates two effects: (i) how much adaptive ensemble behavior differs from LSTM, and (ii) how much performance is protected from weaker graph-only behavior. The threshold column shows that L2 decisions use lower optimal risk cutoffs than D0/L1 in aggregate. GAT underperforms in

L1/L2 mainly through precision loss while recall remains near-saturated (0.9981 in L1 and 0.9990 in L2), indicating over-sensitive triggering when dependency graphs carry weaker cost-discriminative detail than sequence telemetry.

TABLE VII
DOMAIN-LEVEL DELTA ANALYSIS FOR ENSEMBLE F1.

Domain	ENS−LSTM	ENS−GAT	ENS thr.
D0 Synthetic	−0.0166	+0.1494	0.390
L1 TravisTorrent	0.0000	+0.1404	0.371
L2 BitBrains	0.0000	+0.0846	0.180

C. Cross-Domain Stability

Table VIII and Fig. 3 show low seed dispersion in L1 and L2. Ensemble F1 standard deviation is 0.0011 in TravisTorrent and 0.0017 in BitBrains; synthetic remains higher at 0.0122 but still bounded. BWT mean and standard deviation are both 0.0 across all seeds.

TABLE VIII
SEED STABILITY AND ADAPTIVE STRATEGY BEHAVIOR (10 SEEDS).

Domain	F1 std	Min–Max F1	Strategy usage
Synthetic (D0)	0.0122	0.8717–0.9178	9 weighted / 1 lstm
TravisTorrent (L1)	0.0011	0.9059–0.9098	10 lstm
BitBrains (L2)	0.0017	0.9579–0.9637	10 lstm

D. Why the Ensemble Improves Generalization in Practice

The key outcome is not that weighted fusion always wins numerically; the key outcome is that adaptive fusion prevents unnecessary degradation. In D0, weighted combination is selected for 9/10 seeds and contributes balanced precision-recall behavior. In L1 and L2, validation calibration repeatedly favors LSTM-only routing, so the controller avoids injecting lower-quality graph scores into final decisions. This switch is intentional rather than accidental: retain multi-view gains when useful, and preserve robust single-branch behavior when graph evidence is less reliable.

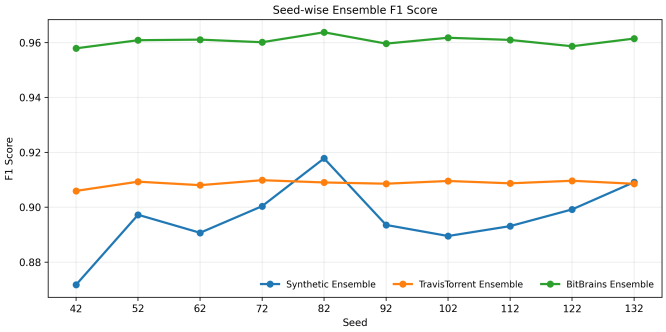

Fig. 3. Seed-wise ensemble F1 trends across the ten-seed protocol.

E. Per-Seed Transparency

Table IX reports ensemble F1 values for every seed and domain, showing that variation is bounded and no single seed dominates conclusions. Fig. 4 and Fig. 5 provide threshold-sweep evidence consistent with the aggregate interpretation.

TABLE IX
PER-SEED ENSEMBLE F1 ACROSS DOMAINS.

Seed	D0	L1	L2
42	0.8717	0.9059	0.9579
52	0.8972	0.9093	0.9608
62	0.8907	0.9080	0.9611
72	0.9004	0.9098	0.9601
82	0.9178	0.9090	0.9637
92	0.8935	0.9086	0.9596
102	0.8895	0.9096	0.9617
112	0.8931	0.9087	0.9610
122	0.8992	0.9096	0.9586
132	0.9091	0.9085	0.9614

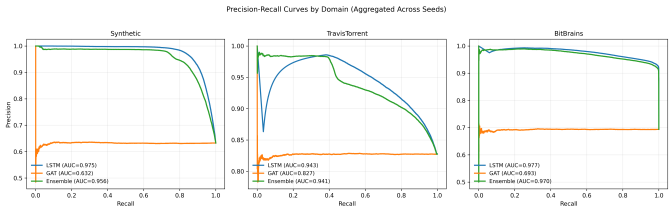

Fig. 4. Precision-recall behavior across domains; L2 maintains high precision at high recall.

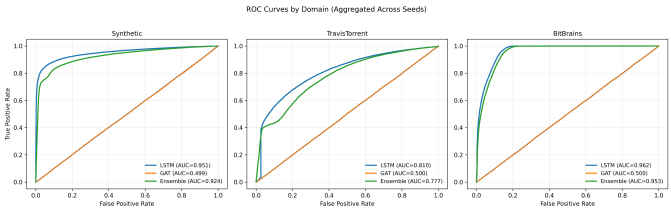

Fig. 5. ROC profiles by domain. L2 shows strongest overall separability.

F. Operational Reliability Evidence

Logs and manifests record repeated early-stop convergence at patience 5, VRAM flush events between model phases, and seed-isolated artifact writes. Combined with immutable aggregate generation, this reduces the risk of hidden cross-seed contamination and improves reproducibility confidence.

VII. PRACTICAL IMPLICATIONS

Cost savings potential. CostGuard shifts response from after-the-fact variance reporting to stage-time intervention, allowing teams to stop high-risk execution patterns before retries and downstream jobs compound spend.

Release risk reduction. Policy gates make release behavior explicit and deterministic. High CRS values or risky context combinations can block continuation automatically, reducing late-stage surprises and reliance on ad hoc manual approval.

CI/CD decision automation. OPA-backed actions (`WARN`, `AUTO_OPTIMISE`, `BLOCK`) automate recurring governance decisions while preserving traceability through stored thresholds, matched rules, and policy source metadata. Representative runtime scenarios are summarized in Table X.

Why engineering teams would adopt this system. CostGuard supports incremental rollout (monitor-only, then warn, then enforce) and preserves operational evidence needed for incident review, compliance audits, and policy tuning without retraining the model for every governance update.

TABLE X
REPRESENTATIVE GOVERNANCE SCENARIOS IN COSTGUARD RUNTIME.

Context	Risk condition	Expected action
Protected branch prod deploy from PR	Context rule hit	BLOCK even at low CRS
Standard stage with CRS in [0.50,0.75)	Warn threshold crossed	WARN + team notification
Sensitive stage above cost ceiling	Cost plus context signal	AUTO_OPTIMISE + remediation hint
CRS $\geq$ 0.90 in any stage	Block threshold crossed	BLOCK and halt pipeline progression

VIII. LIMITATIONS AND THREATS TO VALIDITY

Dataset scope. The three-domain protocol improves diversity, but it does not span all enterprise pipeline topologies, pricing contracts, or release governance cultures.

Domain shift. Calibration and strategy routing can change with new runner types, billing schemas, or workload mixes; periodic recalibration and policy retuning are still required for sustained accuracy.

Hardware constraint bias. Reported behavior is collected under a 4-GB VRAM profile with bounded-memory preprocessing. This makes reproducibility realistic for commodity hardware, but it can bias architectural choices and training dynamics relative to higher-memory deployments.

Model structure limitations. Graph contribution is domain-sensitive; in two domains, adaptive routing prefers LSTM-only decisions. This improves robustness today, but it also indicates that richer graph features or dependency semantics are needed for stronger cross-domain graph utility.

IX. CONCLUSION

CostGuard demonstrates that cost anomaly detection becomes operationally meaningful when integrated with policy-as-code and deployment control. Under a fixed ten-seed, three-domain protocol, the system achieved stable high performance (up to 0.9606 F1 and 0.9763 PR-AUC), complete trial completion, and auditable decision traces on commodity hardware. The central contribution is an accountable control loop where model outputs are not merely observed but converted into enforceable, reviewable pipeline actions. This closes a practical gap between ML experimentation and day-to-day FinOps governance in CI/CD environments.

REFERENCES

[1] C. Balestra, F. Huber, A. Mayr, and E. Müller, "Unsupervised Features Ranking via Coalitional Game Theory for Categorical Data," arXiv:2205.09060, 2022.

[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. ICLR*, 2018.

[4] S. Mukherjee and S. Günnemann, "GhostLink: Latent Network Inference for Influence-Aware Recommendation," in *Proc. CIKM*, 2019.

[5] A. Zareian, S. Karaman, and S.-F. Chang, "Weakly Supervised Visual Semantic Parsing," in *Proc. CVPR*, 2020.

[6] R. Sridivya, M. S. Vijayabaskar, and K. Vimaladevi, "Machine Learning-Driven Improvements in Software Delivery Pipelines," *J. Theor. Appl. Inf. Technol.*, vol. 103, no. 19, 2025.

[7] N. Lokiny and P. Reddy, "Cost Optimization Strategies for DevOps Deployments in Cloud Environments Leveraging Machine Learning," *European J. Adv. Eng. Technol.*, vol. 8, no. 3, pp. 69–72, 2021.

[8] N. M. K. Koneru, "Optimizing CI/CD Pipelines for Multi-Cloud Environments: Strategies for AWS and Azure Integration," *Eastasouth J. Inf. Syst. Comput. Sci.*, vol. 2, no. 3, pp. 288–310, 2025.

[9] J. R. M. Thason and S. K. Bahl, "Cloud Technology and DevOps: A Symbiotic Relationship," *J. Emerg. Technol. Innov. Res.*, vol. 12, no. 3, 2025.

[10] L. R. Pereira, "The Impact of CI/CD in Business Value," M.Sc. thesis, Aalborg Univ., 2025.

[11] S. Deochake, "Cloud and AI Infrastructure Cost Optimization: A Comprehensive Review of Strategies and Case Studies," 2025.

[12] S. Deochake, "ABACUS: A FinOps Service for Cloud Cost Optimization," 2025.

[13] F. Klinaku, S. S. Stie, A. Hakamian, and S. Becker, "An Architectural View Type for Elasticity Policies—The Slingshot Approach," 2024.

[14] Z. Zhang and I. Matta, "SERFLOW: A Cross-Service Cost Optimization Framework for SLO-Aware Dynamic ML Inference," 2025.

[15] A. M. Kirubakaran *et al.*, "Governing Cloud Data Pipelines with Agentic AI," 2025.

[16] G. N. Junior and C. Marcon, "OPTIC: OPtimized Translation and Interaction for Cloud-Costs Using FOCUS Standardization and LLM Integration," *IEEE Access*, 2025.

[17] Open Policy Agent, "Open Policy Agent Documentation," 2024. [Online]. Available: https://www.openpolicyagent.org/docs/latest/

[18] FinOps Foundation, "FinOps Open Cost and Usage Specification (FOCUS)," 2025. [Online]. Available: https://focus.finops.org/

[19] T. S. Kamble, "CostGuard repository and experiment artifacts," 2026. [Online]. Available: https://github.com/tejasskamble/costguard-pade-finops